

DIP Final Project

313510171 郭家均

313581015 趙苡棋

Problem Formulation

In this task we're given multiple images, and the goal is to segment the water body from the picture. There are only 80 images given for training. The output should be a binary picture with water segment white and non-water segment black.

Propose flow

The overall processing flow can be split into 3 steps.

1. Image preprocessing
2. Segment image into regions
3. Evaluate regions probability of being water
4. Output prediction mask

There are many algorithms and different approaches to the first step. The main problem lies in the second step: How do we evaluate each region?

We implemented 3 types of methods.

1. Conventional approach

For the conventional approach, we can not extract any ground true mask feature. We can only derive certain assumptions so that the probability of correct prediction can increase.

Assume water's B channel and G channel are greater than R channel

Assume that water region has smoother surface

2. SVM approach (ML)

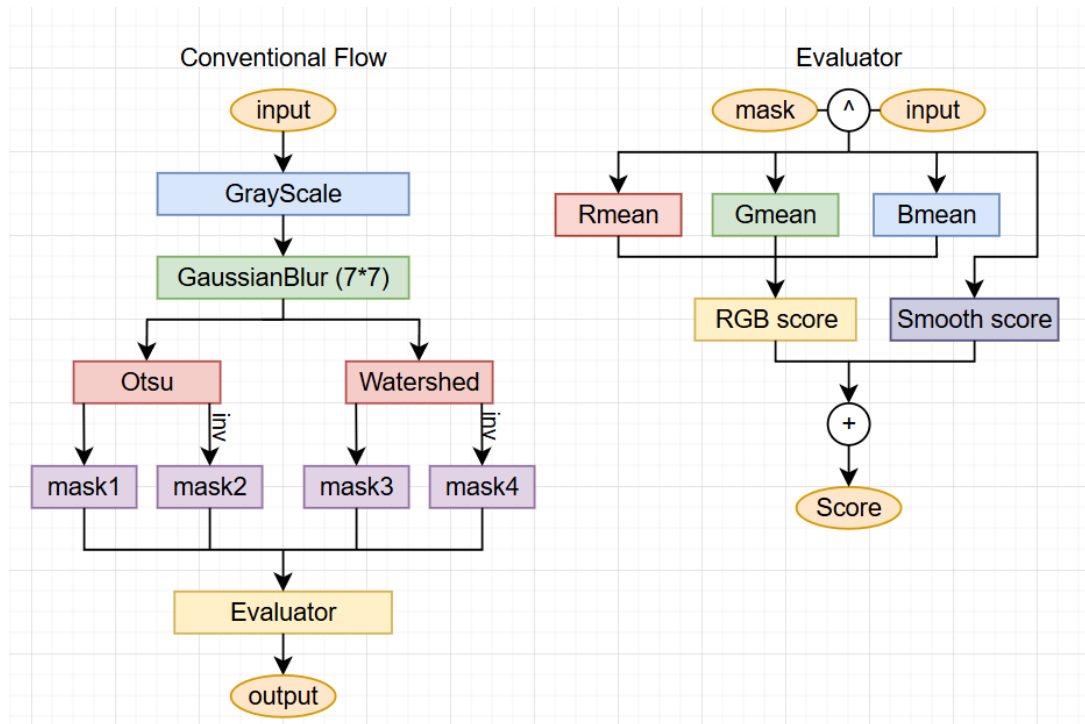
In this section, we can utilize ground truth masks to find more complex decision boundaries using multi-dimensional features.

3. Deep Learning approach

With insufficient training data, we can't use models with high capacity. The regions and masks found in CNN features are harder to comprehend. Besides, the training data has to be representative to the whole dataset.

The following sections further explained details on each method.

Conventional Approach



During the preprocessing stage, we turn images into grayscale and apply gaussian smoothing to reduce noise influence on the following steps.

After preprocessing, we apply Otsu's method and Watershed algorithm to get two masks. For more variety, we apply inverse to each mask to acquire a total of 4 masks for evaluation. Otsu's mask, Inverse Otsu's mask, Watershed mask, Inverse Watershed mask.

During the evaluation stage, we use each mask and input image to do bitwise-AND operation. After acquiring unmask pixels, we compute RGB score and smoothness score.

$$RGB_{score} = (2 * B_{mean} + G_{mean}) / R_{mean}$$

Smoothness score is attained by compute the magnitude of sobel filter on X direction and Y direction for each pixel. Then, compute the mean of every pixel.

$$Smooth_{score} = mean(\sum magnitude(Sobel(X), Sobel(Y)))$$

Combining the two scores we can choose the mask with the highest score as our output.

Achieve average 0.5 IOU in 80 training_dataset image

Only 0.38 IOU in 20 testing_datatset image

=> Performance is not stable enough, the cost function is hard to distinguish between sky and water.

Limitations of Otsu's Method in Water Segmentation

Otsu's method assumes that the pixel values in an image can be divided into two distinct classes (foreground and background) and determines the optimal threshold by maximizing the between-class variance of the pixel intensity histogram. However, this assumption often fails in water segmentation due to the following reasons:

1. Overlap Between Water and Non-Water Pixel Values

Variations in water surfaces, such as waves and reflections, lead to uneven pixel intensities. Also, non-water regions, such as the sky, shadows, and sand, often have pixel values similar to water, resulting in frequent misclassification.

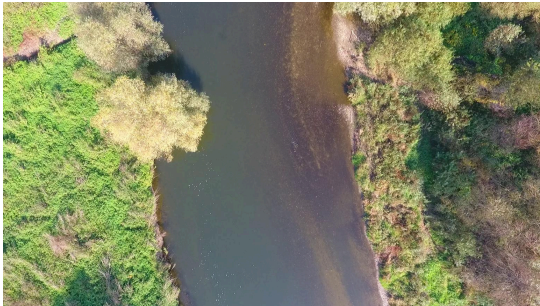
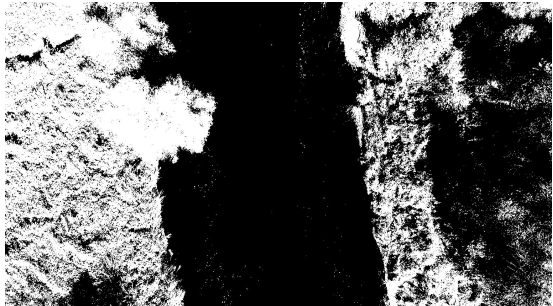
2. Limitations of a Global Threshold

Otsu's method identifies a single global threshold, which cannot adapt to brightness variations caused by lighting conditions or positional differences in the image, such as bright water under direct sunlight and dark water in shaded areas.

3. Binary Classification Assumption

The method is designed for binary classification but water segmentation often involves multiple types of regions, such as water, sand, trees, and sky. This prevents Otsu's method from capturing the complexity of multi-class scenes.

Due to overlapping pixel values, the limitations of a global threshold, and the binary classification assumption, Otsu's method is insufficient for effective water segmentation. More flexible approaches, such as local thresholding methods or advanced deep learning techniques, are required to address these challenges.

Original Image	Output Image
	

An example of Otsu's method failing to segment water regions accurately, as it cannot definitively determine whether the segmented areas are water or non-water. Additionally, the overlapping pixel intensities between water and non-water regions, such as shadows or reflective surfaces, result in significant misclassification.

Challenges and Limitations of the Watershed Segmentation Method

In our experiments with the Watershed segmentation method, we identified a significant issue: while the algorithm is capable of dividing an image into meaningful segments, it frequently misclassified opposite or unrelated regions as water. For example, areas with strong textures or colors resembling water, such as blue skies or shadowed regions, are often incorrectly categorized as water bodies. This misclassification leads to highly inaccurate results, particularly with IoU (Intersection over Union) scores being exceptionally low.

To address this issue, we attempted to reduce the RGB values in highly bright areas of the image, aiming to minimize the influence of these regions when the algorithm distinguishes between the foreground and background. However, this adjustment did not completely resolve the problem.

Limitations of Watershed:

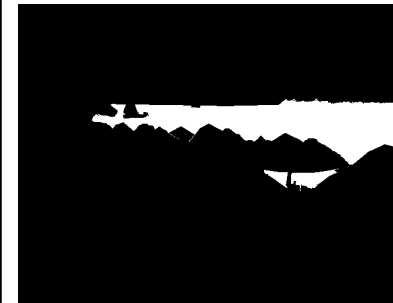
1. Lack of High-Level Features

Watershed relies solely on pixel intensity and gradient information, without leveraging higher-level features such as texture, color consistency, or spatial relationships.

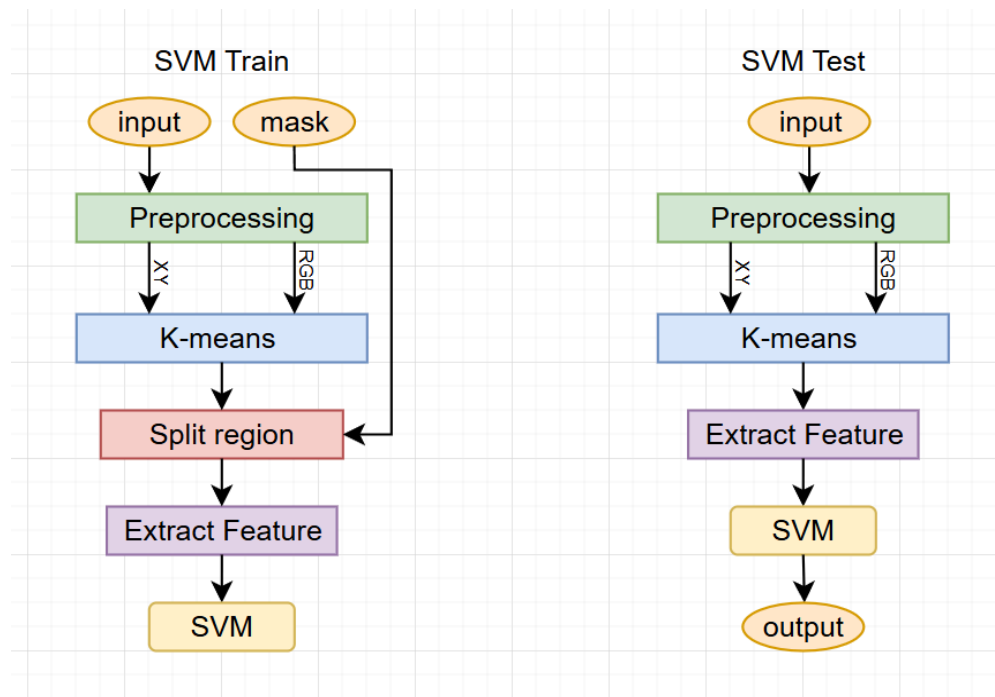
2. Limited Generalization Ability

Due to its lack of contextual understanding, Watershed struggles to generalize effectively across diverse scenes.

These limitations demonstrate that Watershed alone is insufficient for accurate water segmentation. To improve classification performance, it is necessary to incorporate additional feature extraction methods or advanced techniques, such as deep learning, to better address the challenges of this task.

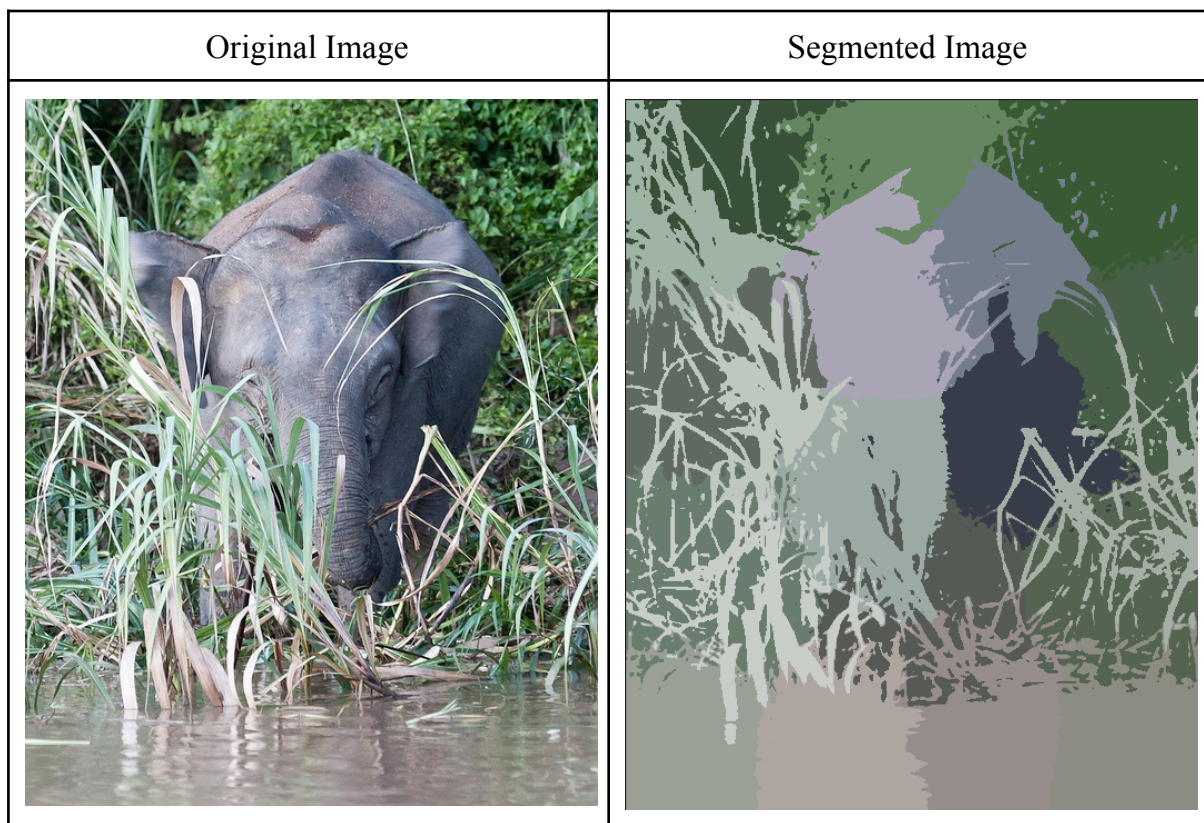
Original Image	Output Image	Mask Image
		

SVM Approach



<Training Stage>

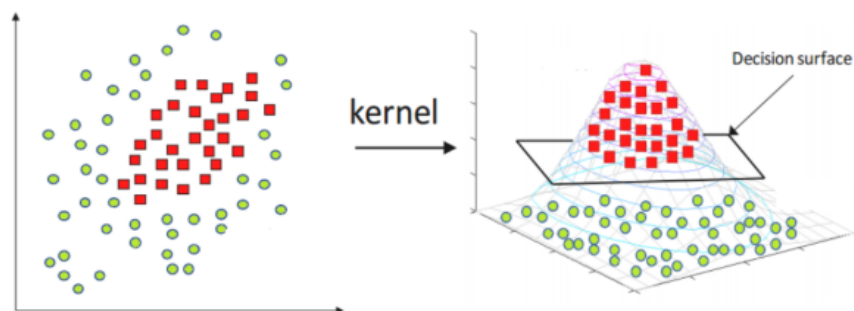
We use the Gaussian Blur kernel to filter the noise, then we apply K-means to separate each image into 20 regions using pixel's [R, G, B, x, y].



The reason to include pixel coordinates is to let pixels nearby have a higher chance of clustering to the same region. After this step, we got a list of every region pixels' coordinates. Next, we use its ground truth mask to further partition the regions into regions purely water and non-water.

We use [Rmean, Bmean, Gmean, contrast, homogeneity, energy, correlation] this vector as our features to train SVM models using polynomial kernels.

We can think of SVM as a mathematical approach to select a decision boundary with maximal margin value on two sets of data points. The kernel is a transfer function that projects the data points to a different dimension so that the decision boundary can easily separate the data.



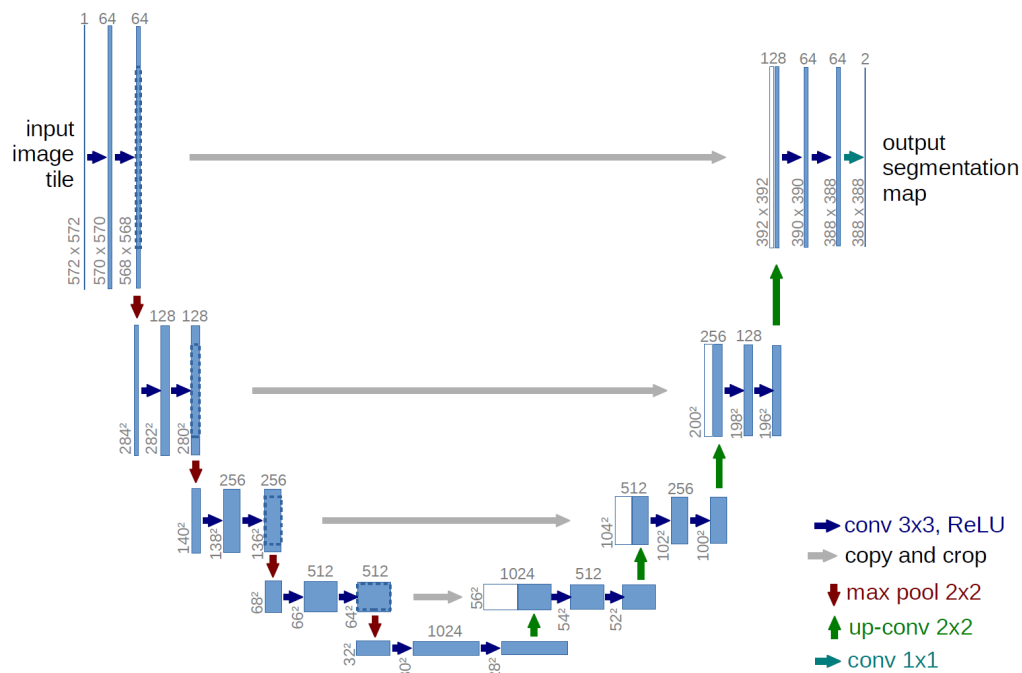
<Testing Stage>

We feed testing images through Gaussian Blur and the same K-means separator. Then we compute the same 7-dimensional feature vector for each region, then assign each region to either water or non-water using SVM models.

We achieved 0.48 IOU in the first testing_dataset and 0.42 on demo testing dataset. This version can distinguish between sky and water more easily.

Original Image	Predicted Mask
	

Deep Learning Approach



Using the DL network has a challenging part of very few training images (only 80 images). We chose a simple Unet structure to implement it.

At first, we resize the training image and mask to 512*512 size. Then we do data augmentation based on random crop, random rotation(within 15 degrees), and K-fold cross validation to prevent overfitting.

The optimizer we’re using is AdamW, which is a regularization version of the typical Adam optimizer. Furthermore, we include a scheduler to reduce learning rate if the training loss increases for 3 epochs in a row.

Unet structure contains short connections to ensure original features can be concatenated to later stages. (no gradient vanish). This method greatly relies on the type of image that was given. For the demo images, this model works poorly on demo images for about 0.29 IOU, but a lot better on another testing set of 0.54 IOU.

Since this method is not stable enough, we don't choose this method.

Comparison

Method	Conventional Approach	Machine Learning (SVM)	Deep Learning (U-Net)
Approach	Uses basic assumptions (e.g., RGB and smoothness) to evaluate water regions.	Utilizes supervised learning with extracted features to classify regions.	Employs a U-Net model with data augmentation and cross-validation.
Strengths	Simple, interpretable, and computationally inexpensive.	Effective at distinguishing between water and non-water regions using handcrafted features.	Captures high-level features and spatial context for segmentation.
Weaknesses	Struggles with overlapping intensities and lacks robustness; misclassifies sky as water.	Relies heavily on the quality of features and SVM's limitations with complex boundaries.	Requires significant data; unstable results due to small training dataset.
Performance (IoU)	0.50 on training, 0.38 on testing.	0.48 on training, 0.42 on testing.	0.54 on training, 0.29 on demo testing.
Generalization	Poor generalization due to reliance on simple heuristics.	Better generalization than the conventional method but limited by feature selection.	Poor generalization due to overfitting on a small dataset.
Ease of Implementation	Easy to implement with basic image processing techniques.	Moderate complexity, requiring feature engineering and SVM tuning.	High complexity, requiring a deep learning framework and careful tuning.

Conclusion

The SVM-based approach balances complexity, interpretability, and performance. With better feature engineering and more training data, it could improve further. Future work should explore hybrid methods or transfer learning with pre-trained models for better generalization.

Reflection

This project has proven to be very challenging. Initially, we believed that using conventional methods to achieve an average IOU of 0.5 would be sufficient. However, after the release of the testing data, we realized that our approach performed well only on the training data, indicating it lacked generalization. We then explored ML and DL methods but encountered similar issues. Due to the small dataset, handling unseen images proved difficult. Despite these challenges, we gained valuable insights throughout the project. Although the performance did not meet our expectations, we developed several alternative solutions to address the problem.